Assessments for JavaScript

Variables, Datatypes, Basics

1. **What is a variable?**

Ans:- A variable is a container with name which holds some data. In JavaScript we can create variables using keywords **var, let** and **const.**

2. **What is the difference between var, let and const?**

Ans:- In JavaScript **var, let** and **const** are keywords. These keywords are used to create variables in JavaScript. Where variables created using var keyword will follow global scope and variables created using keywords **let** and **const** keyword will follow block scope means variables created using **let** and **const** inside a block such as loops or conditional statement can be accessed inside the block itself not outside.

Specially all variables created using **var, let** and **const** keyword inside a function block can be accessed inside the function itself because it follows functional or local scope.

Note:- *Variables created using var keyword inside the block like loops or conditional statement can be accessed outside of the block because it will be inside the global scope and variables can be accessed from global scope.*

Note:- *Variables created using let and const outside of any block will follow script scope. Here the variables can be accessed after the initialization only not before initialization. But variables created using var keyword outside of block can be accessed before its declaration because it is hoisted (moved to the top of the block of scope).*

3. **What are primitive datatypes?**

Ans:- In JavaScript **Primitive datatypes** refers to value based datatype or it can store only one value inside a variable. It is immutable in nature means can't be changed. There are total 7 **primitive datatypes** in latest JavaScript.

a. **Number:-** It represents any integer or decimal value up to 16 digit.
b. **String:-** It represents a sequence of character enclosed within double quotes (“”) or single quotes (‘’) or backticks (``)
c. **Boolean:-** It represents true or false values.
d. **Null:** It's the representation of an empty object or empty value.
e. **Undefined:-** Something is there but not defined

f. **Symbol**

g. **Bigint:-** It can store a value more than 16 digit (integer)

4. **What is typeof?**

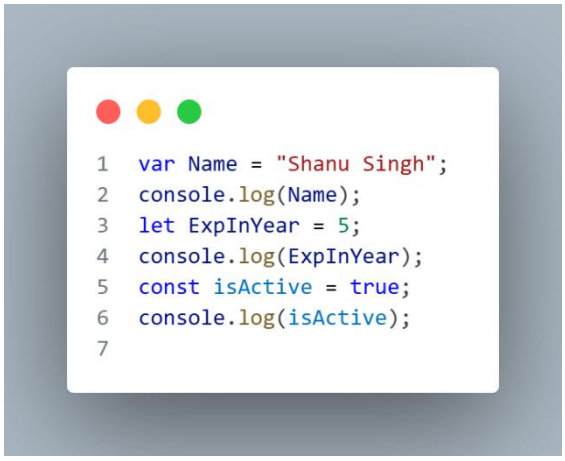
Ans:- In JavaScript typeof is an operator and used to check which kind of datatype is present in any variable. It returns the type of the data present in a variable.

5. **What is undefined?**

Ans:- Undefined is a primitive datatype and it is also represented as a value. It is something which is not defined. If we declare a variable but not initialized then in the created variable by default the value will be stored as undefined.

6. **Declare variables for an employee (Name, Experience, isActive).**

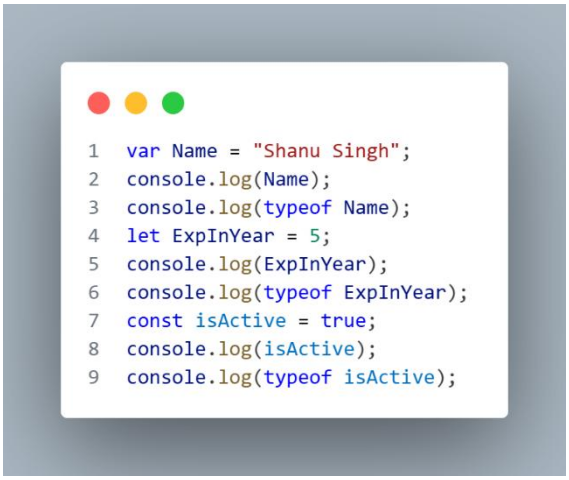
Ans:-



```
1  var Name = "Shanu Singh";
2  console.log(Name);
3  let ExpInYear = 5;
4  console.log(ExpInYear);
5  const isActive = true;
6  console.log(isActive);
7
```

7. **Print the type of Each variable using typeof?**

Ans:-



```
1  var Name = "Shanu Singh";
2  console.log(Name);
3  console.log(typeof Name);
4  let ExpInYear = 5;
5  console.log(ExpInYear);
6  console.log(typeof ExpInYear);
7  const isActive = true;
8  console.log(isActive);
9  console.log(typeof isActive);
```

Strings

1. **What is a string?**

Ans:- String is a primitive datatype in JavaScript. It is a sequence or series of characters enclosed within double quotes or single quotes.

2. Difference between “”, ‘’ and template literals (``).

Ans:- In JavaScript we can create string data in three ways using double quotes, single quotes or template literals. Using double quotes we can write a sentence or word where we need to use apostrophe, we can use single quotes where we need to use single quotes in a sentence or word such as direct or indirect speech. We can use template literals if we want to use JS variables inside a string.

3. What is string immutability?

Ans:- In JavaScript string is a primitive datatype means single value based and primitive datatypes are immutable in nature means once a value is assigned we can’t change the value of the string. We can reassign the value but we can’t change.

4. What is string concatenation?

Ans:- In JavaScript string concatenation represents sum of two string datatype or sum of any datatype with string will be concatenated with the string value.

5. What is a template literal?

Ans:- In JavaScript a template literal is a way to create a string using backticks. Using template literals we can declare multiline string data. It also allow us to embed any variables which has been declared inside a string using this ${variable name}.

Array

1. What is an array?

Ans:- In JavaScript **Array** is a non-primitve datatype. It can hold multiple values in a single variable in the form of indexes. There may be two types of array Homogeneous and heterogeneous. If the same kind of datatype is present inside an array it is homogeneous else it is heterogeneous.

Note:- *Arrays can be modified. It is a reference based datatype. It is a collection of multiple elements.*

2. How do you access elements in an array?

Ans:- To access an element we have some built in methods also by which we can access the elements of an array. Generally we can access the element of an array using their index value. Each element is identified by its position starting from index 0 to positive index only.

3. What is an index in an array.

Ans:- Index is nothing but position of an element of an array. Each element is having index in any programming language starting from 0 to positive index. Index will be always array length -1.

4. **Can arrays store different datatypes?**

Ans:- In JavaScript we can store any kind of datatype in an array together. This is known as **Heterogeneous** array.

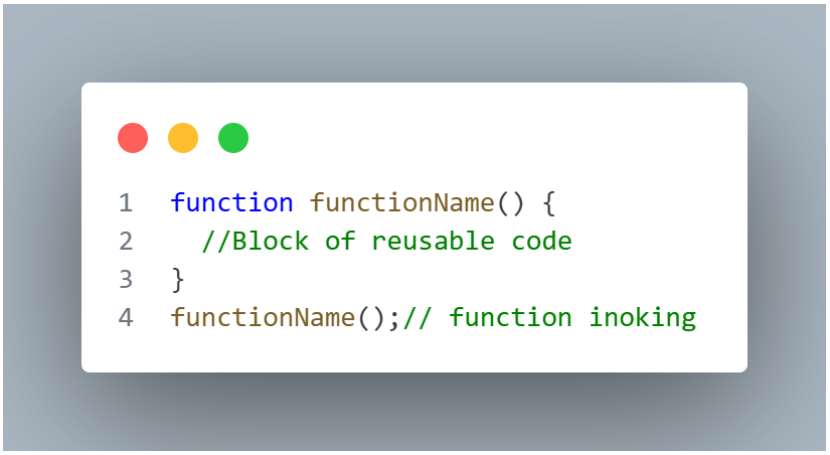
5. **What happens when you access an out of bound index?**

Ans:- If we try to access the element of an array which is not present in an array means out of bound index then it will return me undefined.

Functions

1. **What is a function?**

Ans:- A function is a set of block of reusable code which is used to perform some specific task. We can create a function in JavaScript using function keyword followed by function name followed by parantheses and followed by block of codes. This is also known as Function declaration or Named Function.



```
1  function functionName() {
2    //Block of reusable code
3  }
4  functionName();// function inoking
```

2. **What is the difference between function declaration and function expression?**

Ans:- Function declaration is also known as Named function in JavaScript. It can be hoisted means we can call the function before its declaration.

Function Expression is a type of function in JavaScript which is stored inside a variable. It can't be hoisted.

3. **What is anonymous function?**

Ans:- A function declared without a name is known as anonymous function in JavaScript generally it can be used in higher order and callback function.

4. What is an IIFE?

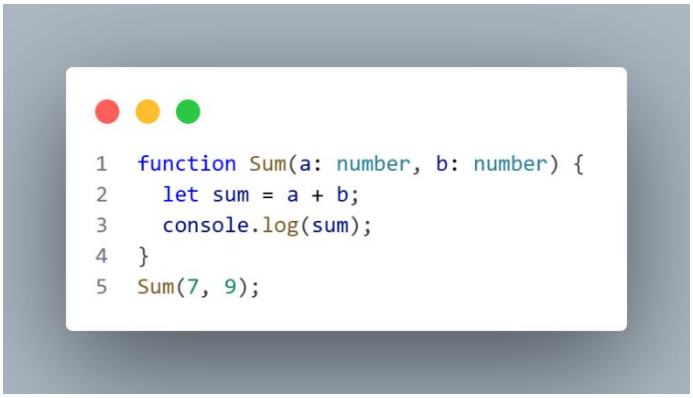
Ans:- An IIFE stands for Immediately Invoked Function Expression. This function will be executed only once in the entire webpage lifecycle. It executes as soon as it has been declared.

5. What is hoisting?

Ans:- Hoisting in JavaScript is a mechanism where JavaScript moves the variables to top of their current scope. In JavaScript Named function and variables created using var keyword are hoisted.

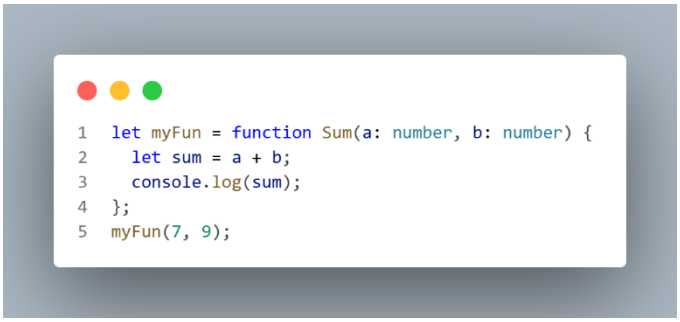
6. Write a function to add two numbers?

Ans:-



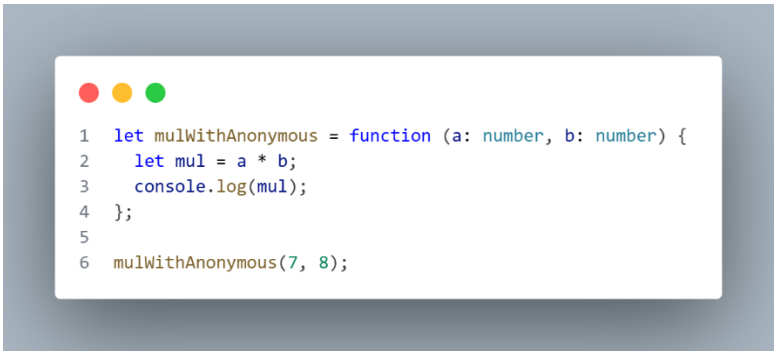
7. Convert Function declaration into function Expression.

Ans:-



8. **Write an anonymous function to multiply two numbers.**

Ans:-

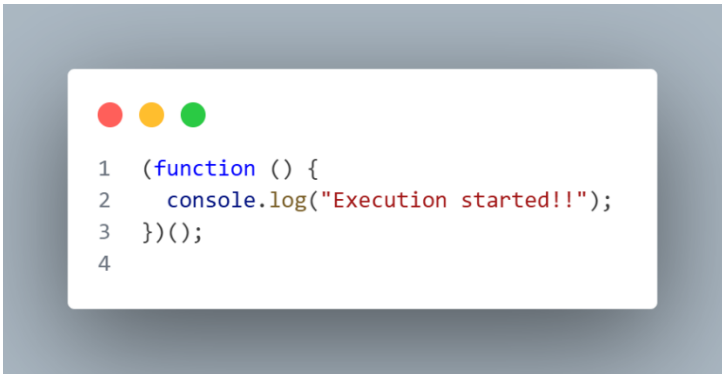


```
let mulWithAnonymous = function (a: number, b: number) {
  let mul = a * b;
  console.log(mul);
};

mulWithAnonymous(7, 8);
```

9. **Write an IIFE that prints execution started.**

Ans:-



```
(function () {
  console.log("Execution started!!");
})();

```

Scope and Hoisting

1. **What is scope in JavaScript?**

Ans:- Scope in JavaScript represents the visibility or accessibility of a variable. There are three types of scope Global, block and local or functional. Variables created using var will be in global scope, variable created using let and const follows block scope and inside a function variables created using var let and const will follow local or functional scope. Global variable can be accessed any where in the script file. Block scope variable can be accessed inside the block itself such as loops or conditional statement. Local scope variables can be accessed inside the function itself.

Note:- *Variable created using let and const globally will follow script scope means it can be accessed after the initialization only.*

2. **What is global scope?**

Ans:- Global scope represents the visibility or accessibility of a variable which has been declared globally. It can be accessed any where in the script file once after declaration.

3. What is function scope?

Ans:- Function scope represents the visibility or accessibility of a variable which has been declared inside function. It is also known as local scope. The variables will be accessed inside the function block itself not outside.

4. What is block scope?

Ans:- A block scope represents the visibility or accessibility of a variable declared inside the block such as loops or conditional statement. The variables created using let and const will follow the block scope means variables created using let and const keyword inside the block will be accessed inside the block itself.

Variables created using var keyword inside the block will be accessible outside of the block also because it follows global scope.

5. Which variables follow block scope?

Ans:- Variables created using let and const keyword only will follow block scope. Variables created using var keyword does not follow block scope.

6. Are variables hoisted?

Ans:- Yes, The variables are hoisted to the top of their containing scope in JavaScript but initializations are not hoisted. In JavaScript variable hoisting may work different with different variables. Variables declared using var can be hoisted to the top of their scope and by default **undefined** value will be assigned to the variable. If we talk about variables declared using **let** and **const** keyword then it will be waiting in **TDZ (**Temporal Dead Zone**).** If we try to access these kind of variables then it will give us reference error.

TDZ (Temporal Dead Zone)

TDZ stands for Temporal Dead Zone, it is an area of block where a variable is inaccessible until the moment the computer completely initializes it with a value. The variables created using let and const will be waiting here until it is initialized.

7. Are Functions hoisted?

Ans:- Yes, Functions are hoisted and the entire function block will be moved to top of their scope. In JavaScript only **Named Function** or **Function declaration** is hoisted means we can call or invoke it before its declarations.

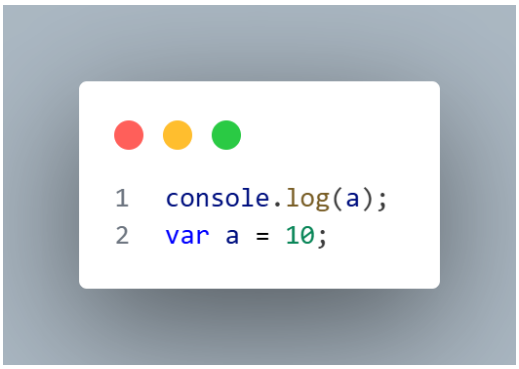
8. Difference between hoisting of var, let and const?

Ans:- Variables are hoisted in JavaScript whether it is created using var, let or const no matter. But all these variables created using these keywords have different aspect. Variables created using var keyword will be hoisted to top of its scope and here the undefined value will be assigned by default before its initialization.

Variables created using let and const keyword also hoisted but it will wait inside Temporal Dead Zone until it gets initialized.

9. Predict Output:

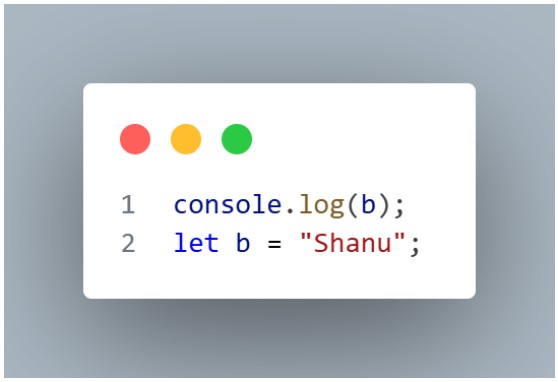
Ans:-



Here in the above image in JavaScript it will print undefined because the variable has been declared using var and it follows global scoping and initialized with undefined. We are trying to access before the initialization so in variable a before initialization the value will be undefined so it will print undefined.

10. Predict Output:

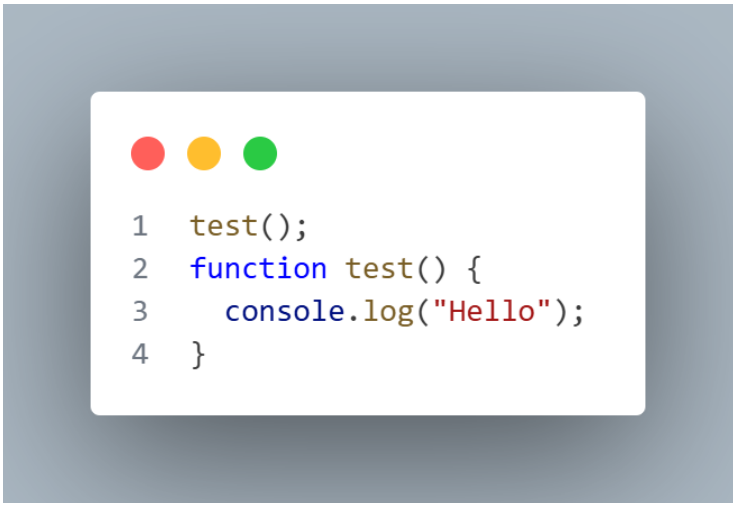
Ans:-



Here in the above code will return reference error because the variable has been created using let keyword and it will follow script scope where the value will be unavailable until and unless it is initialized.

11. Predict Output:

Ans:-

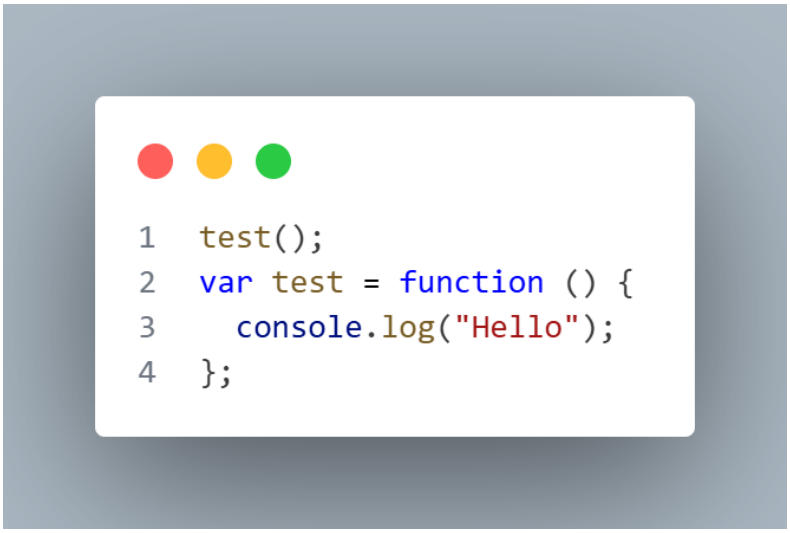


```
1  test();
2  function test() {
3    console.log("Hello");
4  }
```

Here, in the above code it will print hello because whenever we are talking about function declaration the entire function will be moved to the top of their scope. The function will be executed without any error.

12. Guess Output:

Ans:-



```
1  test();
2  var test = function () {
3    console.log("Hello");
4  };
```

In the above code it will throw us error like test is not a function. Here we have declared the variable using var keyword means it will be also hoisted along with the value undefined and we are trying to call the function with undefined value so it will throw the error test is not a function.